Conceptual Thinking

Functional & Non-Functional Requirements,
User stories & Use Case Description

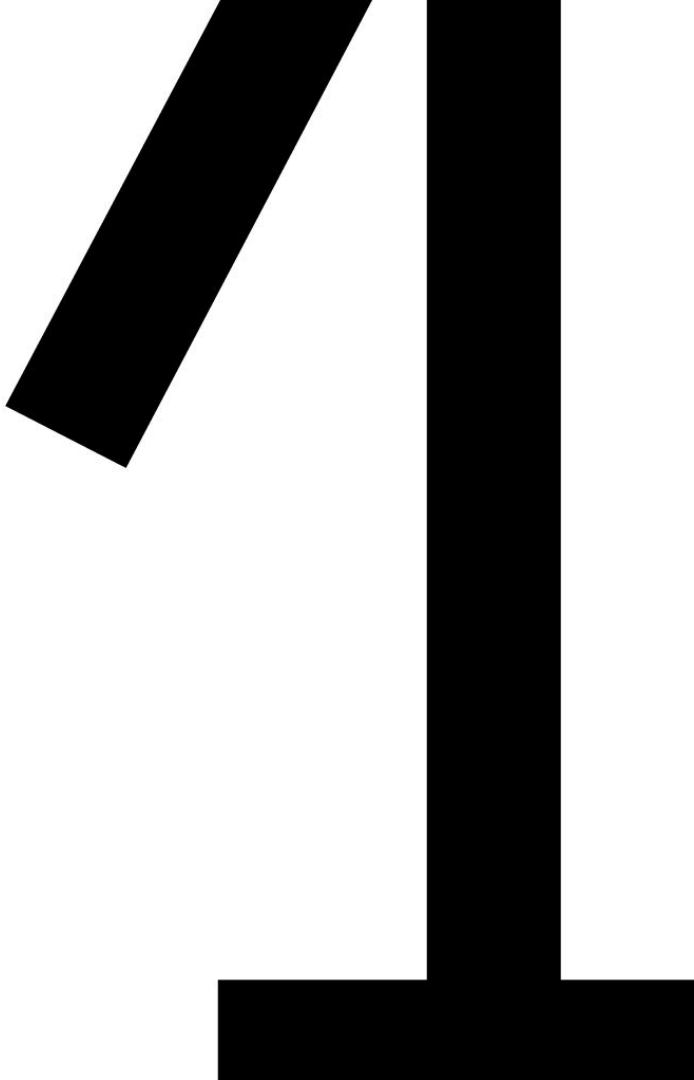
KdG Karel de Grote Hogeschool

Content

1. Functional vs. Non-functional Requirements
2. User stories & epics
3. Use cases description
4. User stories vs use cases descriptions

KdG
Karel de Grote Hogeschool

Functional vs. Non-functional Requirements



Requirements

- The behavior of a system is described in requirements

- **Requirement/need =**

  something that a system/project must meet

Two main types of requirements...

Functional Software Requirement

A functional software requirement is behavior (functionality) that the system must exhibit

=> something the system can do

e.g.: adding a new customer to the system
e.g.: A customer should be able to place an order through the online store.

Non-Functional Software Requirement

A non-functional software requirement is a quality standard that the system must meet

=> the way the system can do something

e.g.: the system must be available in both Dutch and English
e.g. usability: The system should allow users to complete a purchase in no more than three steps.
e.g.example Reliability: The system must have an uptime of 99.9%.
e.g. performance: The system must respond to user actions within 2 seconds.

More specific...

FURPS

-Functional: Capability (Size & Generality of Feature Set), Reusability (Compatibility, Interoperability, Portability), Security (Safety & Exploitability)

-Usability (UX): Human Factors, Aesthetics, Consistency, Documentation, Responsiveness

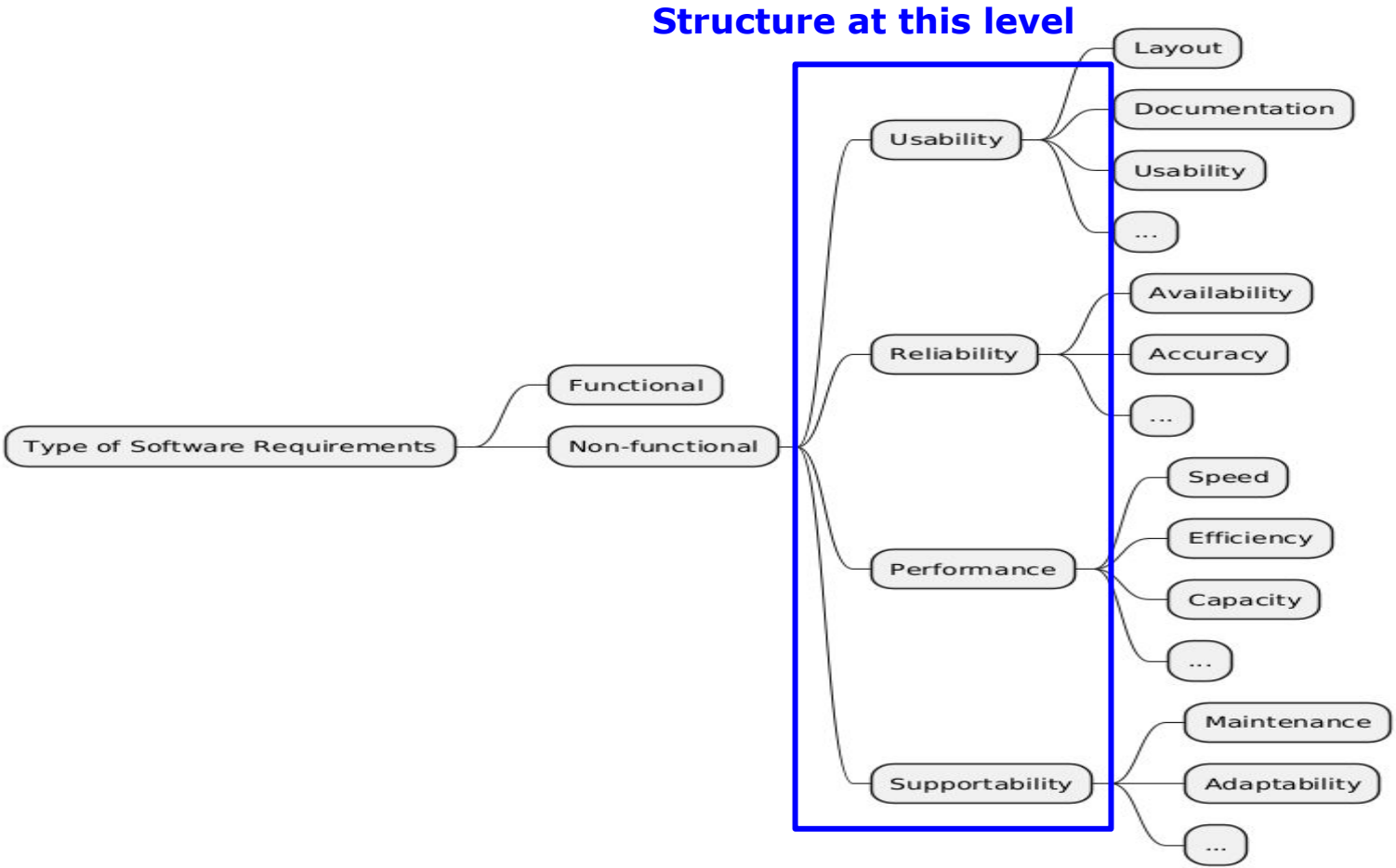
-Reliability: Availability (Failure Frequency (Robustness/Durability/Resilience), Failure Extent & Time-Length(Recoverability/Survivability)), Predictability (Stability), Accuracy (Frequency/Severity of Error)

-Performance: Speed, Efficiency, Resource Consumption (power, ram, cache, etc.), Throughput, Capacity, Scalability

-Supportability (Serviceability, Maintainability, Sustainability, Repair Speed): Testability, Flexibility (Modifiability, Configurability, Adaptability, Extensibility, Modularity), Installability, Localizability

More info: https://en.wikipedia.org/wiki/FURPS

FURPS schematically represented



Sample exercise: assignment

FURPS NMBS

A ticket machine and associated software are being developed for a railway company. For each of the following requirements for this product, indicate which category in the FURPS model it belongs to.

1.After confirmation of purchase, the ticket must be available within 3 seconds.

2.The ticket machine must be available 98% of the time.

3.The ticket machine must complete 99.9% of all purchases without errors.

4. The ticket machine must sell single and return tickets from the station where it is located to all stations in Belgium.

5. Users should be able to choose a map type and destination with no more than five touches on the screen.

Example exercise: solution

FURPS NMBS

A ticket machine and associated software are being developed for a railway company. For each of the following requirements for this product, indicate which category in the FURPS model it belongs to.

1.After confirmation of purchase, the ticket must be available within 3 seconds.

Performance

2.The ticket machine must be available 98% of the time.

Reliability

3.The ticket machine must complete 99.9% of all purchases without errors.

Reliability

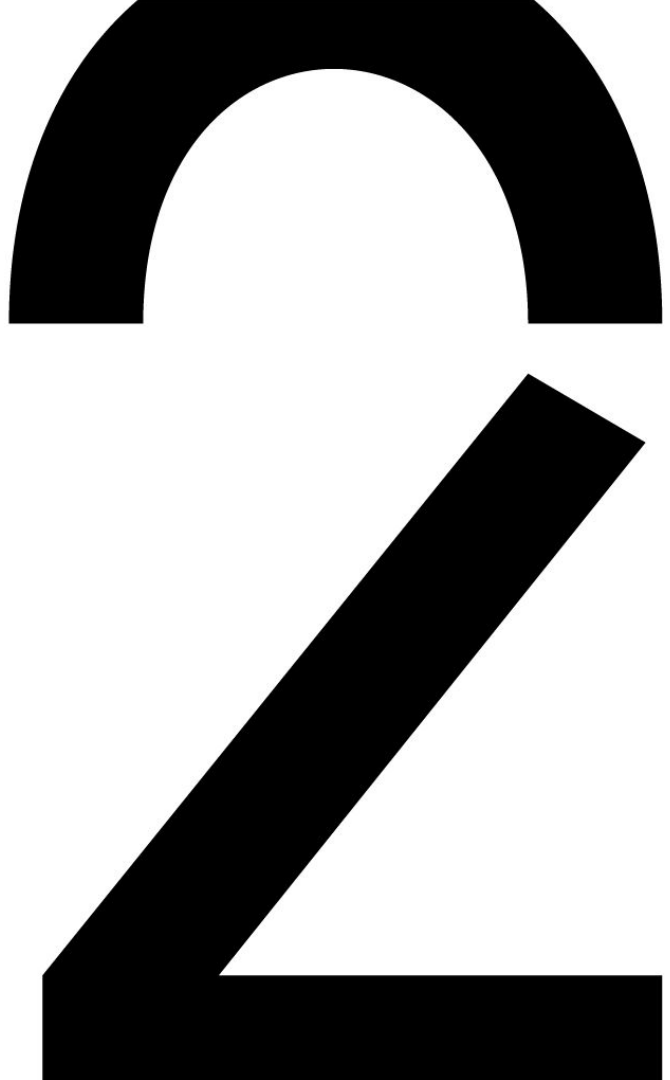
4. The ticket machine must sell single and return tickets from the station where it is located to all stations in Belgium.

Functional

5. Users should be able to choose a map type and destination with no more than five touches on the screen.

Usability

User stories & epics



Requirements

In every IT project, needs and requirements must be formulated. This must be done for both new products and new features.

Requirements gathering, or requirements capture, is the process of trying to understand the problems you're trying to solve and/or the needs you're trying to fulfill. This is done by talking to a selection of current users as well as potential users.

There are several ways to do this, we'll cover two of them.

Requirements gathering

1. User stories & epics
2. Use cases

User story

User stories are short, simple descriptions of features, told from the perspective of the user or customer of the system who wants a (new) functionality. They represent the **outcome** of a functionality, not the way it works internally.

The structure is usually like this

As a <type of user>,

I want <some goal>

so that <some reason>

The "so that" part is optional

The goal of a user story isn't to focus on how to build it, but on who wants the feature, what it will do, and why it's important.

User stories & epics

User stories can be written at various levels of detail. A user story can encompass a large amount of functionality and is then generally known as an epic.

Example of an epic user story:

As a user

I want to backup my entire hard drive,

so that I don't lose my files.

User stories & epics

Because an epic is generally too large, it is split into several smaller user stories before being worked on:

As a power user

I want to I specify files or folders to backup based on file size, date created, and date modified

As a user

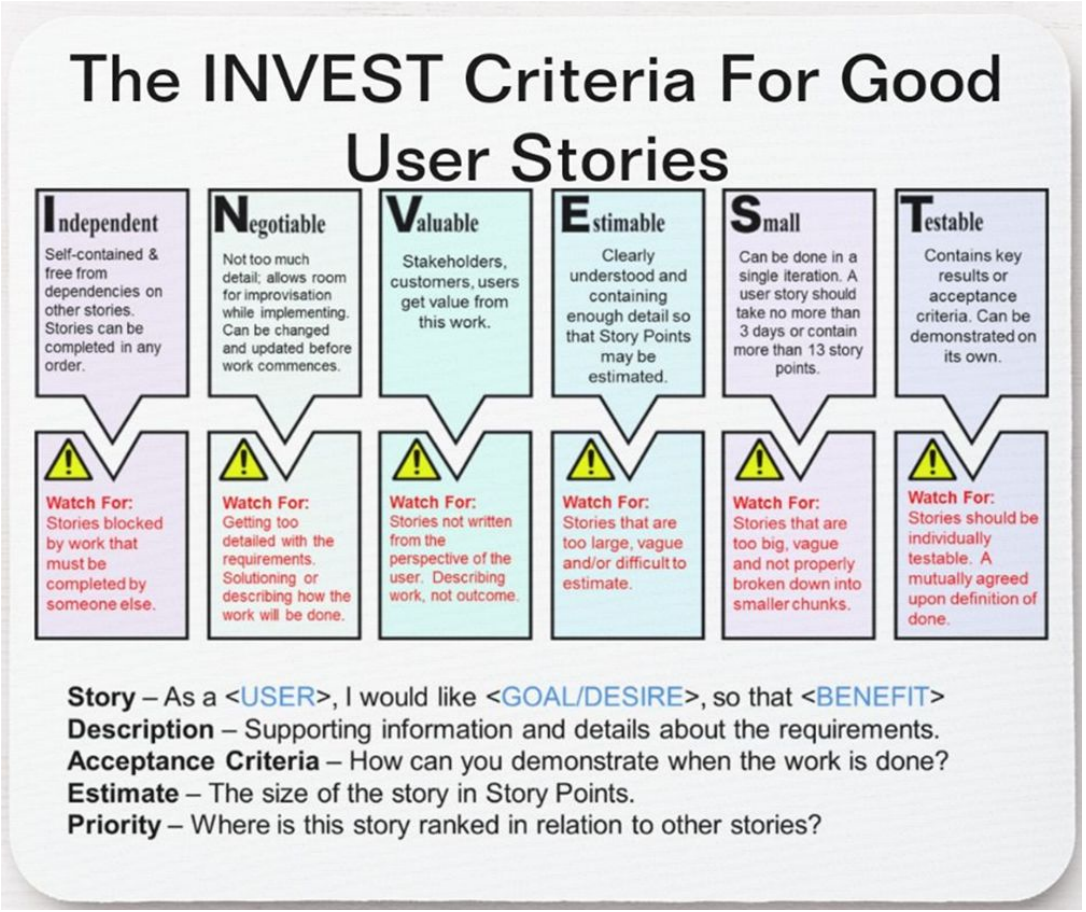
I want to specify folders or files within folders to back up, so that my backup drive is filled with the files I want saved

As a user

I want to specify folders that should not be backed up

so that my backup drive doesn't get filled up with files I don't need to save.

User stories & epics



Examples of user stories & epics

User story	Acceptance criteria
As a user of an auction website, **I need to** select an auction product in the website's ordering platform **so I can** bid on it.	Ensure that the user of the auction website is able to: ● to log in to the website ● navigate to the auction page ● select a product(s) to bid on
As a user of the auction website, **I need to** be able to view my previous bids on the platform **so I can** delete expired bids.	Ensure that the user of the auction website is able to: ● to log in to the website ● navigate to an auction page to view previously bid items ● select one or more expired bids ● delete expired bids

Acceptance criteria => more information in SE2

Acceptance criteria are based on the user story and describe the factors that ensure a test passes. In other words, acceptance criteria can be used to verify when a user story, after implementation in the product, is complete and working as expected.

Acceptance criteria are a formalized list of requirements that ensure that all user stories are completed and all scenarios are considered.

Given - When - Then ⇒ GWT

Given - When - Then ⇒ GWT

= Gherkin

Use cases description

Use cases

Use cases are one of many ways to write down functional requirements

Use case = a use of the system

Determining the size of a use case:

a use case corresponds to a basic (business) process that happens:

by one person,

in one location,

at one moment!

Use cases

- Still analyzing…
  - So: modeling requirements, WHAT a system should be able to do/do
- …but is very close to design (HOW a system does something), but such statements try to avoid this
  - For example, avoid “display on tablet”, but say generically “system displays”

- **Important aspects:**
  - **Completeness: Are all possible situations and exceptions included? This is crucial for use cases!**
  - **Ambiguity: Can everything be interpreted in only one way?**

- Show the interaction between actor(s) and the system

Use case description

UC descriptions are not a UML standard, so use your own template (available here):

Use Case name	
Goal	
Actors	
Preconditions	
Postconditions	
Trigger event	
High Succes Scenario (HSS)	1. 2. 3. 4. 5. 6. 7. 8. 9. 10.
Alternative scenarios	

Use case description: all components explained

Component	Explanation	Example
Use Case Name	A short, clear title that describes what the system does for the actor. It should be action-oriented and specific.	Withdraw Cash from ATM
Goal	Describes the main objective the actor wants to achieve through this interaction with the system.	"Allow the customer to withdraw cash from their bank account using an ATM."
Actor	**Entities that interact with the system. These can be:** ● **Primary actor: initiates the use case to achieve a goal.** ● **Supporting actor: helps the system complete the use case.** ● **Off-stage actor: affected by the use case but not directly interacting.**	Primary: Customer - Off stage: ATM Machine - Supporting: Bank System (verifies PIN & debits account)
Precondition	States what must already be true before the use case begins. Ensures the system is in a valid state to execute the use case.	"Customer must have a valid bank card."
Postcondition	Describes the state of the system after the use case is completed successfully.	"Cash is dispensed, customer has card back, and the account is debited."
Trigger Event	The event or action that starts the use case. Often initiated by the primary actor.	"Customer inserts their bank card into the ATM."

Use case description: all components explained

Component	Explanation	Example
Main Success Scenario (MSS) or Happy Flow	The step-by-step flow of how things go perfectly. Each step = one action or interaction. Limit to maximum 10–15 steps.	1. ATM requests PIN 2. Customer enters PIN 3. ATM reads card data 4. ATM sends PIN and account to Bank 5. Bank verifies PIN 6. ATM prompts for amount 7. Customer enters amount 8. ATM sends request to Bank 9. Bank approves 10. ATM dispenses cash 11. ATM returns card 12. Customer takes card and cash
Alternative Scenarios	Describes exceptions or variations in the flow. Could be system errors, wrong inputs, or optional paths. Each should be clearly numbered or labeled.	• **Alt 1: Invalid PIN → ATM displays error, allows 2 more tries.** • **Alt 2: Insufficient balance → ATM shows balance and cancels transaction.**

Use case description: points of interest

- Make sure there is no overlap between trigger and first step in MSS
  - The trigger provides the direct cause for the start of the MSS
  - You can have multiple triggers for the same UC, so a clear distinction is needed
- A UC description is only as good as the completeness of all extensions/exceptions
  - Many extensions/exceptions make the system more robust (can handle more situations)
- **Guideline: maximum 10 to 15 steps in the main success scenario or alternative. Including sub-steps!**
  - If more: split steps into separate use case

Use case description: points of interest

- **Precondition vs. alternative: how to provide for "exceptions"?**
  1. Alternatively, you include everything that is outside the system's influence
     - What should the system not assume will proceed smoothly?
     - For example: person has insufficient cash on hand when paying
  2. As a precondition you include everything that absolutely must be fulfilled before starting the MSS
     - **Without a precondition being met, the MSS cannot start**
- **Postconditions "minus" preconditions = content of the use case/what happens**
- In Actors you only mention the actors themselves, not the system/subsystem

Types of actors

Actor = a kind of "user" of the system. Someone/something who interacts with the system in the broadest sense. This is a role, not a specific person. It can be a human actor or another system/device.

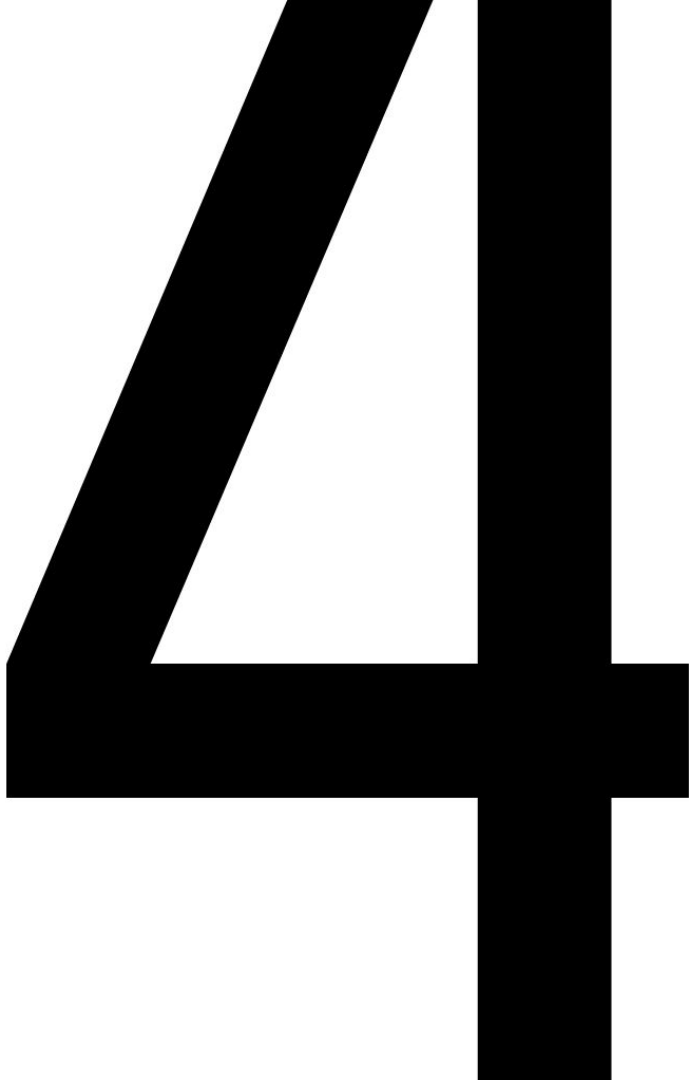
Types of actors, to be reviewed in order:

1. **primary actor (P): 'starts' the use case, is often (not always!) the most important actor**
2. **supporting actor (S): is necessary to execute the use case, so performs actions in the use case**
3. **offstage actor (O): (usually) gets something from the use case such as information, an object,... Does not perform any actions, only receives things.**

Extra: System under Development (SuD)

The system or subsystem for which we are writing out functionality in the form of use cases

User stories vs use cases descriptions



3 Similarities Between UCs and USs

1. Description from the user's perspective
One of the key similarities is that both describe a desired functionality of a system from the user's perspective. The focus is on business value without delving into technical details.

2. Elements to determine the scope of a system
Both can be used to define the functional and technical boundaries (scope) of a system. For a scope definition, a general description of the functionality and responsibilities is sufficient.

3. Elements to determine the size of the system
Both can be used to estimate the size of a system or release. Using only use cases provides a more abstract (higher) estimate than using an extensive collection of user stories.

Use cases and user stories can also be used to complement each other.

3 differences between UCs and USs 1/2

1. User stories are primarily used for planning
User stories originated within the Extreme Programming (XP) methodology, where they were primarily used as a "playing piece" in the planning game. All user stories are typically placed on the backlog, providing a prioritized list that the Product Owner and development team can use to plan the next iteration or release.
For each individual story, the necessary tasks (development, testing, requirements, etc.) are determined that are needed to deliver the desired functionality.

2. Use cases are primarily used for structured requirements recording
Use cases were developed to structurally describe all possible ways a user (actor) can achieve a goal. We can capture many requirements and desires in a single use case, expressed as user requirements.

Regardless of the depth applied, the primary goal of use cases is to structurally record the requirements and wishes imposed on a system.

3 differences between UCs and USs 2/2

3. Use cases are valuable as reference material
When using user stories, a new user story is usually written for each change without updating the old one. This can make it difficult to maintain a clear picture of the current functionality after delivery.

Because use cases are primarily intended for the structured recording of requirements, they can be used as a reference if maintained correctly.

Additional information about user stories vs. use cases

https://www.visual-paradigm.com/guide/agile-software-development/user-story-vs-use-case/

https://www.digitalnatives.hu/blog/user-story-vs-use-case/

https://medium.com/@a.reskova/the-difference-and-relationship-between-use-case-and-user-story-25e24df777a3

https://uxmag.com/articles/user-stories-vs-use-cases-how-they-stack-up

Recurring Example: Monopoly

Purpose of the Exercise

This exercise is an ongoing case study in Conceptual Thinking. Each week, we apply what we've learned to a digital version of Monopoly.

Why do we choose Monopoly?

- Everyone knows it, or almost everyone
- It contains rich structures: players, money, rules, actions, spaces, cards
- It provides an ideal context for modeling object-oriented systems
- It is tangible and recognizable, making abstract concepts concrete

Recurring Example: Monopoly

Learning objective

Students understand how theory is translated into a working conceptual model by:

- Working iteratively with one consistent case
- Apply different analysis methods (use cases, class diagrams, ...)
- Gain insight into the relationship between models and system behavior

Quick Refresher: How Does Monopoly Work Again?

- 2–6 players move their pawn around the board with a roll of 2 dice
- Players buy properties they land on
- Other players pay rent when they land on someone else's property
- There are special subjects such as Chance, Community Fund, Tax, Prison, etc.
- Players can take out mortgages, trade properties, or go bankrupt
- Goal: Become the richest player by bankrupting others

Glossary

- **Player: A participant in the Monopoly game.**
- **Board: The physical/visual representation of the Monopoly game.**
- **Property: A buyable location on the board.**
- **Bank: The system that manages money distribution.**
- **Dice: Random generator to determine movement.**

Object Oriented Analysis (1/2)

Objects Identified

- Player – A user who participates in the game.
- GameController – Controls the game flow and manages turns.
- Board – The game board with all the positions.
- Position – A field on the board; can be of different types.
- Property – A purchasable position on the board (specialization of Position).
- Bank – Manages money, mortgages, and ownerless properties.
- Dice – Produces random values from 1 to 6.
- Ownership – The relationship between a player and a property.
- Card – Cards such as “Chance” or “Community Chest”.
- Jail – Special status of a player, blocks actions.

Object Oriented Analysis (2/2)

Object	Responsibilities
Player	Rolls dice, moves around, buys properties, pays/receives money, draws cards, goes to jail, applies for a loan.
Board	Contains and manages all positions; determines starting position; keeps track of each player's location.
Position	Determines behavior when a player lands on it (e.g. tax, buy property, jail).
Property	Stores information about the cost price, rental price, and owner; determines whether a player can buy/pay.
Bank	Distributes money, processes transactions, manages unsold properties, manages loans and foreclosures.
Dice	Generates a random value (1–6), sometimes twice (for double).
Ownership	Keeps track of which player owns which property, with optional mortgage status info.
Card	Contains instructions (e.g. “Go to jail” or “Get $50”) and triggers actions when pulled.
Jail	A condition in which a player may not make normal turns unless released by payment, dice roll, or card.

Functional requirements (1/6)

Method 1: User stories

As a [User/Actor/Archetype]	I want to be able to [do a thing]	so that I can [achieve a goal]
Game Organizer	**Show the game rules**	**understand how to play the game**
Game Organizer	Show information on how to play the ASCII version of the game.	The player knows how the game navigation works.
Game Organizer	Make it possible to add a player.	Players can be added.
Game Organizer	Ask a player for their name.	Playername can be added to a player.
Game Organizer	Ask a player for their pawn color	Playerpawn can be added to a player.
Game Organizer	Make an extra invisible player who will manage the bank.	Bank will be managed.
Game Organizer	Make a Dices object	Players will use this to throw the dices.
Game Organizer	Let the Dices object roll.	Players can use the Dices.
Game Organizer	Show the bankruptcy rules.	The player knows the bankruptcy rules.
Game Organizer	Show the current situation at a given time in the game.	The players know the situation.
Game Organizer	Register when the game has been played.	The application knows when to stop.

Functional requirements (2/6)

As a [User/Actor/Archetype]	I want to be able to [do a thing]	so that I can [achieve a goal]
Application	Define Position object attributes for all types (properties, bank, chance, etc.)	Position attributes are available to fill the board.
Application	For each position, make a Position object with the correct attributes.	Positions are available to fill the board.
Application	Ask each buyable position if it already has been bought.	Apply the right action when a player hits the position.
Application	Ask each buyable position for its price.	Apply the actions when a player hits the position.
Application	Ask each position for legal actions.	Apply the actions when a player hits the position.
Application	Ask each position for its current rent.	Apply the actions when a player hits the position.
Application	Define attributes for the 4 corner positions.	Apply the right action when a player hits the corner position.
Application	Use the Positions to fill the board.	Have a correct board to play with.
Application	Create an ASCII playing board based on the standard Monopoly board.	Update and show it during the game.
Application	Make 2 to 10 players visible on the start position on the board.	Use this as the start situation.
Application	Show the current amount of tax money and jail money after each turn.	All players know the sum they can win.
Player	Enter my name.	The player name has been registered.
Player	Enter my pawn.	The player pawn has been registered.
Player	Buy a property.	I own the property.
Player	Buy a railway station.	I own the railway station.
Player	Receive rent on my properties.	Calculate my current currency position.
Player	Ask the bank for a loan.	I can continue buying properties.
Player	If I am in jail, get out of it.	To continue playing.
Player	When I reach the Go position I want to receive double fee.	To follow the rule.

Functional requirements (3/6)

Methode 2: use cases

Use case 1	Roll Dice
Name	Roll Dice
Goal	Move the player based on a die roll
Actors	Player
Precondities	- It's the player's turn - Game is active
Postcondities	- Player has been moved - New position is being evaluated
Trigger	Player clicks on 'Roll Dice'
Main success path	1. Player chooses 'Roll Dice' 2. Dice generates a value 3. GameController counts total 4. Player is moved 5. New position is being evaluated
Alternative paths	- A1: Player throws double → extra turn - A2: Player lands on 'Go To Jail' → moves to Jail, status = “in jail”

Use case 2	Buy Property
Name	Buy Property
Goal	Let player buy a vacant property
Actors	Player
Precondities	- Player stands on unoccupied property - Player has enough money
Postcondities	- Property added to player - Player's balance has decreased
Trigger	Player chooses the option 'Buy Property'
Main success path	1. GameController detects free ownership 2. The ‘Buy Property’ button appears 3. Player chooses 'Buy Property' 4. Bank validates balance 5. Money is transferred 6. Ownership is registered
Alternative paths	- A1: Player has insufficient balance → purchase declined - A2: Player refuses purchase → property remains unowned

Functional requirements (4/6)

Use Case 3	Explain Rules
Name	Explain Rules
Goal	Show game rules to new players
Actors	Game Organizer
Precondities	- Game has not started yet
Postcondities	- Players have had access to game rules
Trigger	Organizer chooses option 'Explain Rules'
Main success path	1. Organizer selects 'Explain Rules' 2. System shows rules 3. Players read or listen to explanation
Alternative paths	- A1: Players close lines prematurely - A2: Organizer skips lines

Functional requirements (5/6)

F – Functional Requirements

- Players should be able to roll a die and automatically move their pawn on the board.
- Players must be able to buy, sell, and mortgage properties.
- The system should automatically manage turns and indicate whose turn it is.
- Cards (Chance/Community Fund) must be drawn when a player lands on the appropriate space, with the effect automatically executed.
- The game must end when there is only one player left, or if ended manually.

U – Usability

- The interface should be intuitive so that a new player can start a game independently within 5 minutes.
- Game rules should always be accessible via a button during the game.
- The application should be playable on both desktop and tablet without sacrificing user experience.
- Each player should receive clear visual feedback on actions (such as dice, trades, properties).

R – Reliability

- The system must consistently follow the rules of the game, including complex scenarios such as three doubles = jail.
- The application must have at least 99.8% uptime during gaming sessions.
- If a player unexpectedly disconnects, the game state must be preserved and can be resumed.
- Transactions such as payments to the bank or other players must never fail or be duplicated.

Functional requirements (6/6)

P – Performance

- Game moves (such as moving a pawn or buying a property) must respond within 1 second.
- The game should be playable without noticeable lag with up to 6 simultaneous players.
- The application should automatically sync across all involved player screens after a turn update.

S – Supportability

- All game actions (such as turn order, ownership changes, bank transactions) should be logged for debugging or replay purposes.
- New card instructions (Opportunity/Community Fund) should be configurable through an admin panel without any code changes.
- The system must be scalable for expansions such as house rules or other game boards.